Microprocessors Systems – CCEN 316

LAB PROJECT

Microprocessors Based ATM

Students' Names and ID No:

Hasan Al Hussein 100063390

Yaman Masad 100063500

Salaheddine Metnani 100064666

Section: B2

Instructor: Bushra AlShehhi

Submission Date: 02/05/2025

Table of Contents

Abstract

This project presents the design and implementation of a microprocessor-based ATM simulation using the FRDM KL25Z development board. The system utilizes a 4x4 matrix keypad, a push-button switch, and UART communication with a PC terminal (TeraTerm) to simulate user authentication and transaction operations. Users are required to enter a 5-digit PIN through the keypad, with up to three attempts allowed for successful authentication. Upon successful login, the user inputs a withdrawal amount using the PC keyboard. The system validates the amount against a preset limit and handles all scenarios, including incorrect PIN entries, withdrawal limit violations, and input timeouts. Hardware configurations such as keypad interfacing and switch integration, as well as software modules for UART communication, keypad scanning with debouncing, and timeout handling, were successfully implemented. The project demonstrates a complete embedded system workflow, highlighting key microcontroller programming concepts such as GPIO control, real-time event handling, and robust user interaction.

Aim and Objectives

- To design and implement a microprocessor-based ATM simulation using the FRDM KL25Z board, a 4x4 matrix keypad, and a push button.
- To program PIN authentication, validate withdrawal amounts, handle timeouts, and perform UART communication with a PC terminal (TeraTerm).

Introduction

This project aims to simulate an ATM system using the KL25Z microcontroller. The system asks users to authenticate with a 5-digit PIN entered via a matrix keypad. After successful authentication, the user is prompted to enter a withdrawal amount through the PC keyboard. The system handles multiple conditions, including incorrect PIN attempts, exceeding withdrawal limits, and input timeouts. Communication with the user is performed via UART on a terminal emulator (TeraTerm).

Hardware Design

- Microcontroller: FRDM KL25Z (ARM Cortex-M0+).
- Keypad: 4x4 matrix keypad connected to:
  - Rows: PTB0, PTB1, PTB2, PTB3 (configured as outputs).
  - Columns: PTE0, PTE1, PTE2, PTE3 (configured as inputs with pull-up resistors).
- Switch: Connected to PTC12 (starts the authentication process).
- PC Terminal: TeraTerm, connected via USB (OpenSDA interface on KL25Z).

To ensure reliable keypad operation, pull-up resistors were used on the column lines (Port E pins PTE0–PTE3). These resistors hold the lines HIGH when no key is pressed, allowing the microcontroller to detect key presses accurately when a line is pulled LOW. Furthermore, a software debouncing technique was applied to mitigate the key bouncing effect, which causes false or multiple detections when a button is pressed or released. Proper debouncing ensures that only a single input is registered for each key press, improving the stability and reliability of the system.

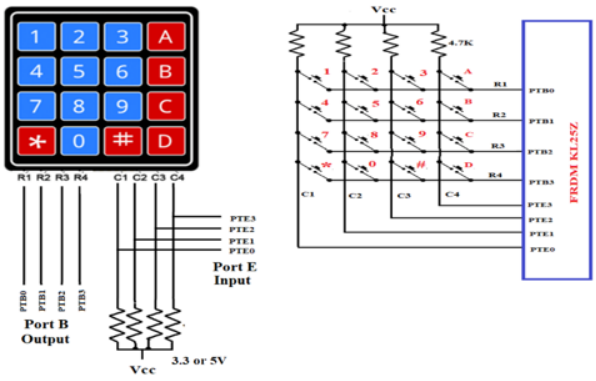


Figure 1: 4x4 Matrix Keypad interfacing with FRDM KL25Z (Rows on Port B, Columns on Port E)

This diagram illustrates the hardware interfacing between the FRDM KL25Z microcontroller and the 4x4 matrix keypad. The keypad rows (R1–R4) are connected to Port B pins (PTB0–PTB3) configured as outputs, while the columns (C1–C4) are connected to Port E pins (PTE0–PTE3) configured as inputs with pull-up resistors. This setup allows the microcontroller to scan key presses by sequentially pulling each row LOW and reading the corresponding column states. The push-button switch used to trigger the PIN entry process is connected to PTC12 in pull-down configuration.

To visually represent the overall structure of the ATM simulation system, the following block diagram summarizes the major components and how they interact with the FRDM-KL25Z microcontroller. It clearly shows the GPIO mappings for the keypad, the UART link to the PC terminal, and the use of a dedicated switch to trigger user input.

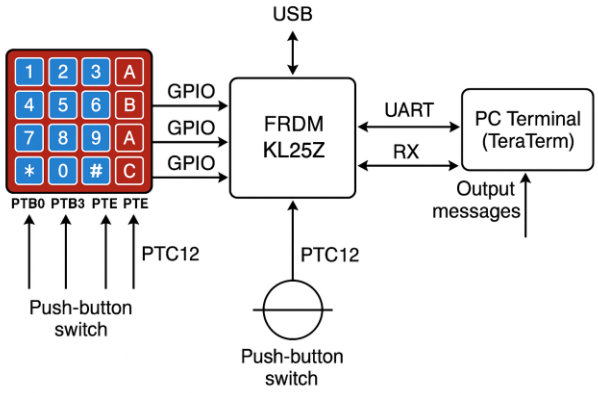


Figure 2: Block diagram of the ATM system using the FRDM-KL25Z microcontroller. The 4x4 matrix keypad is connected via GPIO pins (PTB0–PTB3 for rows, PTE0–PTE3 for columns), and the push-button is connected to PTC12. UART communication is used to interact with the PC terminal (TeraTerm) through PTA1 (TX) and PTA2 (RX).

Software Design

- UART Initialization: UART0 configured for 9600 baud rate communication.
- GPIO Initialization:
  - Keypad rows are set as outputs, columns (with pull-up resistors) as inputs.
  - Switch configured as input.
- Keypad Scanning:
  - Sequentially pull rows low and detect key press via column inputs.
  - Software debounce implemented by delay after key detection.

The following flowchart visualizes the scanning mechanism used to detect key presses on the matrix keypad. It highlights the row-by-row activation strategy, column detection logic, and debounce handling before key confirmation. This process is looped until a key is pressed or the timeout threshold is reached.

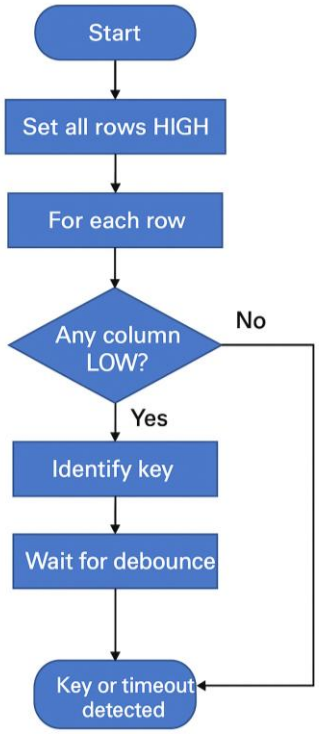


Figure 3: Flowchart of the keypad scanning process. Each row is driven LOW one at a time while monitoring the column inputs (PTE0–PTE3). When a LOW column is detected, the corresponding key is identified. A debounce delay is applied before confirming the key press or timeout.

Port B	PTB3	PTB2	PTB1	PTB0	Port E	PTE3	PTE2	PTE1	PTE0	Key
Row	R4	R3	R2	R1	Column	C4	C3	C2	C1	pressed
0x0E	1	1	1	0	0x0E	1	1	1	0	1
					0x0D	1	1	0	1	2
					0x0B	1	0	1	1	3
					0x07	0	1	1	1	A
0x0D	1	1	0	1	0x0E	1	1	1	0	4
					0x0D	1	1	0	1	5
					0x0B	1	0	1	1	6
					0X07	0	1	1	1	B
0x0B	1	0	1	1	0x0E	1	1	1	0	7
					0x0D	1	1	0	1	8
					0x0B	1	0	1	1	9
					0x07	0	1	1	1	C
0x07	0	1	1	1	0x0E	1	1	1	0	*
					0x0D	1	1	0	1	0
					0x0B	1	0	1	1	#
					0x07	0	1	1	1	D

Figure 4: Bit combinations for identifying key presses using row and column scanning

The table above provides the specific bit combinations read from Port B and Port E when each key is pressed. By setting one row LOW at a time and checking which column reads LOW, the microcontroller can determine exactly which key was pressed. Each key produces a unique row-column pairing, and this mapping is essential for accurate keypad scanning and decoding. The values such as 0x0E, 0x0D, 0x0B, and 0x07 correspond to different row outputs, enabling reliable detection of inputs during PIN entry.

- PIN Authentication:
  - User enters 5 digits through the keypad.
  - System compares with the correct PIN (63390).
- Withdrawal Input:
  - After successful authentication, amount is entered via keyboard.
  - Amount is accepted only if ≤ 5000.
- Timeout Handling:
  - Approximately 5-second timeout for both PIN and withdrawal inputs.
- User Feedback:
  - Messages sent via UART to the TeraTerm terminal.

Verification and Analysis

- Correct PIN: Successful authentication message displayed; withdrawal input requested.
- Incorrect PIN: After 3 incorrect attempts, session is aborted.
- Withdrawal Validation:
  - If amount > 5000 → Message: “Insufficient balance”.
  - If amount ≤ 5000 → Message: “Transaction success”.

- Timeout:
  - No input within 5 seconds results in a “Timeout!!” message and termination.
- ASCII Conversion: Inputs from keyboard are correctly converted from ASCII to integers.

Test Plan

Test Scenario	Expected Result
Correct PIN and withdrawal ≤ 5000	"Transaction success"
Correct PIN and withdrawal > 5000	"Insufficient balance"
Wrong PIN on first try, correct second	"Incorrect Pin. Try again." then "Entered pin is correct"
Wrong PIN three times	"Incorrect Pin. Aborted!"
No PIN input within timeout	"Timeout!!"
No withdrawal input within timeout	"Timeout!!"

The Code:

The following code implements the ATM system functionality, including keypad scanning, PIN authentication, withdrawal processing, UART communication, and timeout handling, based on the FRDM KL25Z platform.

```
#include <MKL25Z4.h>
#include <string.h>
#include <stdlib.h>

#define PIN_LENGTH 5
#define TIMEOUT_COUNT 32 // Timeout for keypad scanning (~5 seconds)

char correct_pin[PIN_LENGTH + 1] = "63390"; // Predefined correct PIN

// Function Prototypes
void UART_init(void);
void GPIO_init(void);
void delay_ms(int n);
void sendString(char *str);
void sendChar(char ch);
char receiveChar_withTimeout(int *timeoutFlag);
```

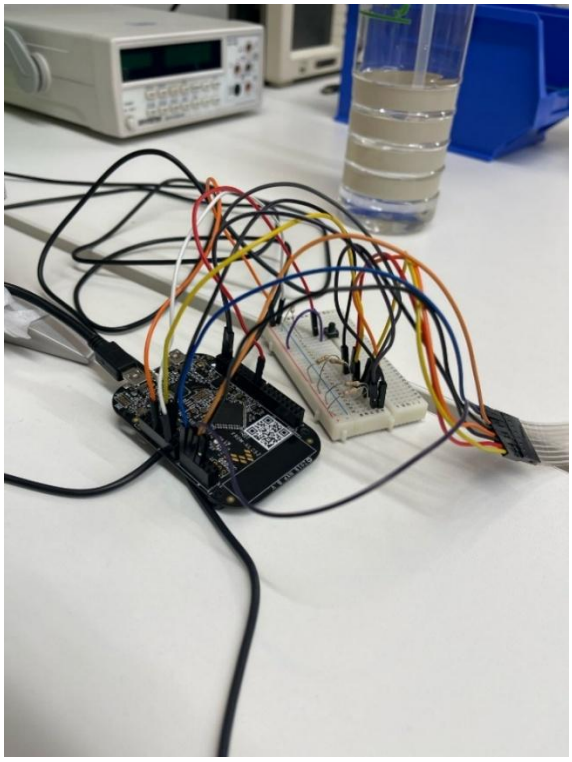
```
char keypad_getkey(void);

int main() {
    UART_init(); // Initialize UART communication
    GPIO_init(); // Initialize GPIO for keypad and switch

    int attempts = 0; // Number of PIN attempts
    int timeoutFlag = 0; // Timeout flag for UART input

    while (attempts < 3) {
        sendString("\r\nPress the button to enter PIN...");

        // Wait for button press
        while (!(GPIOC_PDIR & (1UL << 12))) {}
        delay_ms(20); // Debounce
        // Wait for button release
        while (GPIOC_PDIR & (1UL << 12)) {}
        delay_ms(20); // Debounce

        sendString("\r\nEnter 5-digit Pin number: ");
        char entered_pin[PIN_LENGTH + 1]; // Buffer to store entered PIN

        // Read 5 digits from keypad
        for (int i = 0; i < PIN_LENGTH; i++) {
            entered_pin[i] = keypad_getkey();
            if (entered_pin[i] == 0) { // Timeout during PIN entry
                sendString("\r\nTimeout!!\r\n");
                while (1); // Halt the system
            }
            sendChar(entered_pin[i]); // Echo key pressed
        }
        entered_pin[PIN_LENGTH] = '\0'; // Null-terminate the entered PIN

        // Compare entered PIN with correct PIN
        if (strcmp(entered_pin, correct_pin) == 0) {
            sendString("\r\nEntered pin is correct\r\n");
            sendString("Enter amount to be withdrawn: ");

            char amount_str[5]; // Buffer to store withdrawal amount
            // Read 4 digits of amount
            for (int i = 0; i < 4; i++) {
                amount_str[i] = receiveChar_withTimeout(&timeoutFlag);
                if (timeoutFlag) { // Timeout during amount entry
                    sendString("\r\nTimeout!!\r\n");
                    while (1);
```

```
                }
                if (amount_str[i] < '0' || amount_str[i] > '9') { // Check if input
is digit
                    sendString("\r\nInvalid input! Enter digits only.\r\n");
                    while (1);
                }
                sendChar(amount_str[i]); // Echo entered digit
            }
            amount_str[4] = '\0'; // Null-terminate amount string

            int amount = atoi(amount_str); // Convert ASCII to integer
            // Check balance
            if (amount > 5000) {
                sendString("\r\nInsufficient balance\r\n");
            } else {
                sendString("\r\nTransaction success\r\n");
            }
            while (1); // End program after transaction
        } else {
            attempts++; // Increment wrong attempts
            if (attempts == 3) {
                sendString("\r\nIncorrect Pin. Aborted!\r\n");
                while (1);
            } else {
                sendString("\r\nIncorrect Pin. Try again.\r\n");
            }
        }
    }
}

// UART Initialization
void UART_init(void) {
    SIM_SCGC4 |= SIM_SCGC4_UART0_MASK;   // Enable UART0 clock
    SIM_SOPT2 |= SIM_SOPT2_UART0SRC(1);   // Select MCGFLLCLK as UART clock source
    SIM_SCGC5 |= SIM_SCGC5_PORTA_MASK;    // Enable PORTA clock

    PORTA_PCR1 = PORT_PCR_MUX(2);         // PTA1 as UART0_TX
    PORTA_PCR2 = PORT_PCR_MUX(2);         // PTA2 as UART0_RX

    UART0_C1 = 0x00;                      // 8-bit data, no parity, 1 stop bit
    UART0_C4 = 0x0F;                      // Over sampling ratio 15
    UART0_BDH = 0x00;                     // Baud rate high byte
    UART0_BDL = 0x89;                     // Baud rate low byte (9600 baud)
    UART0_C2 |= UART0_C2_TE_MASK | UART0_C2_RE_MASK; // Enable UART transmitter and
receiver
```

```
}

// GPIO Initialization
void GPIO_init(void) {
    SIM_SCGC5 |= SIM_SCGC5_PORTB_MASK | SIM_SCGC5_PORTE_MASK | SIM_SCGC5_PORTC_MASK;
// Enable clocks for Ports B, E, C

    PORTC_PCR12 = PORT_PCR_MUX(1);        // Configure PTC12 as GPIO (switch)
    GPIOC_PDDR &= ~(1UL << 12);           // Set PTC12 as input

    for (int i = 0; i < 4; i++) {
        PORTB_PCR(i) = PORT_PCR_MUX(1);   // Configure PTB0-PTB3 as GPIO (Rows)
        PORTE_PCR(i) = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK; //
Configure PTE0-PTE3 as GPIO input with pull-up (Columns)
    }
    GPIOB_PDDR |= 0x0F;  // Set PTB0-PTB3 as outputs (rows)
    GPIOE_PDDR &= ~(0x0F); // Set PTE0-PTE3 as inputs (columns)
}

// Keypad Scan
char keypad_getkey(void) {
    const char keymap[4][4] = {
        {'1', '2', '3', 'A'},
        {'4', '5', '6', 'B'},
        {'7', '8', '9', 'C'},
        {'*', '0', '#', 'D'}
    };

    int count = 0;
    while (count < TIMEOUT_COUNT) {
        GPIOB_PSOR = 0x0F; // Set all rows HIGH

        for (int row = 0; row < 4; row++) {
            GPIOB_PCOR = (1 << row); // Drive one row LOW
            delay_ms(20); // Debounce

            int col_val = GPIOE_PDIR & 0x0F; // Read column inputs
            if (col_val != 0x0F) { // Check if any column pulled LOW
                for (int col = 0; col < 4; col++) {
                    if (!(col_val & (1 << col))) {
                        GPIOB_PSOR = 0x0F; // Reset all rows HIGH
                        return keymap[row][col]; // Return detected key
                    }
                }
            }
```

```
            GPIOB_PSOR = (1 << row); // Reset current row HIGH
        }
        count++;
    }
    return 0; // Timeout if no key pressed
}

// Delay Function (~1ms per iteration)
void delay_ms(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < 16000; j++) {}
    }
}

// Send String through UART
void sendString(char *str) {
    while (*str) {
        while (!(UART0_S1 & UART0_S1_TDRE_MASK)) {} // Wait for transmit buffer
empty
        UART0_D = *str++;
    }
}

// Send Single Character through UART
void sendChar(char ch) {
    while (!(UART0_S1 & UART0_S1_TDRE_MASK)) {} // Wait for transmit buffer empty
    UART0_D = ch;
}

// Receive Single Character through UART with Timeout
char receiveChar_withTimeout(int *timeoutFlag) {
    int count = 0;
    while (!(UART0_S1 & UART0_S1_RDRF_MASK)) { // Wait for received data
        delay_ms(1);
        count++;
        if (count > 5000) { // Timeout condition (~5 sec)
            *timeoutFlag = 1;
            return 0;
        }
    }
    *timeoutFlag = 0;
    return UART0_D; // Return received character
}
```

Pictures (Hardware):



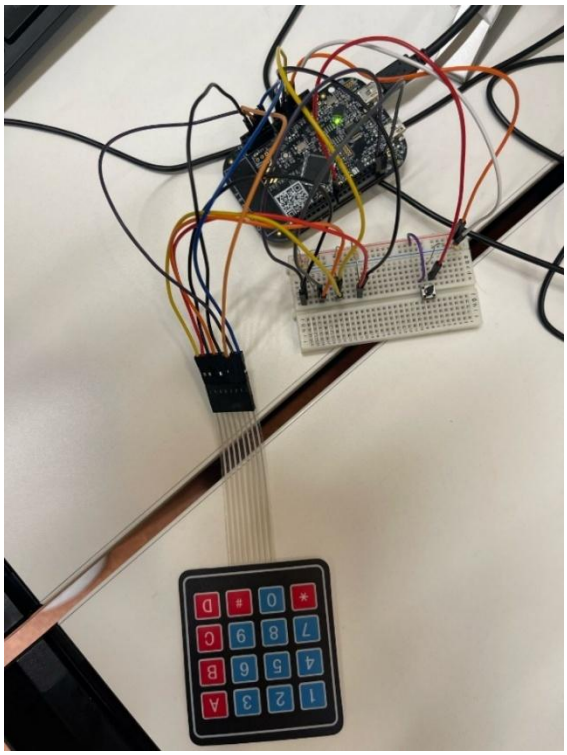


Figure 5: Hardware setup of FRDM KL25Z, keypad, and button connections

Pictures (Test Cases):

Test Scenario	Expected Result
Wrong PIN on first try, correct second	"Incorrect Pin. Try again." then "Entered pin is correct"
Correct PIN and withdrawal > 5000	"Insufficient balance"

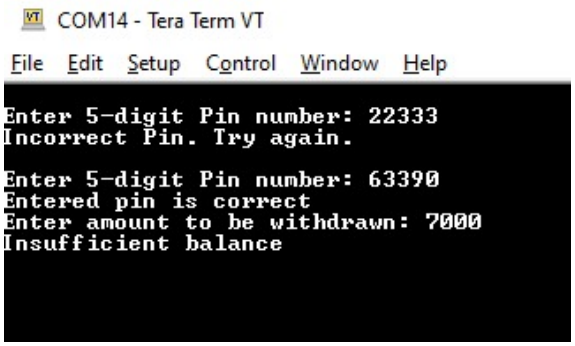


Figure 6: Incorrect PIN first try, correct PIN second try, withdrawal amount exceeds limit, then insufficient balance

Test Scenario	Expected Result
Wrong PIN three times	"Incorrect Pin. Aborted!"

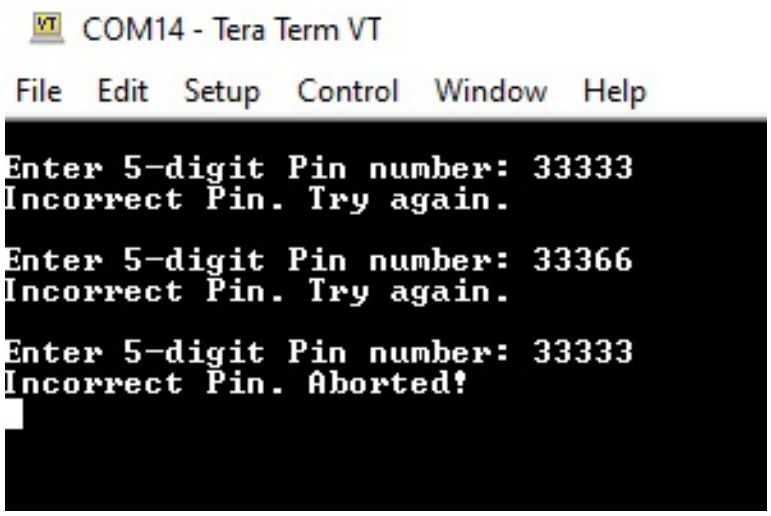


Figure 7: Three incorrect PIN attempts – "Incorrect Pin. Aborted!"

Test Scenario	Expected Result
Correct PIN and withdrawal ≤ 5000	"Transaction success"

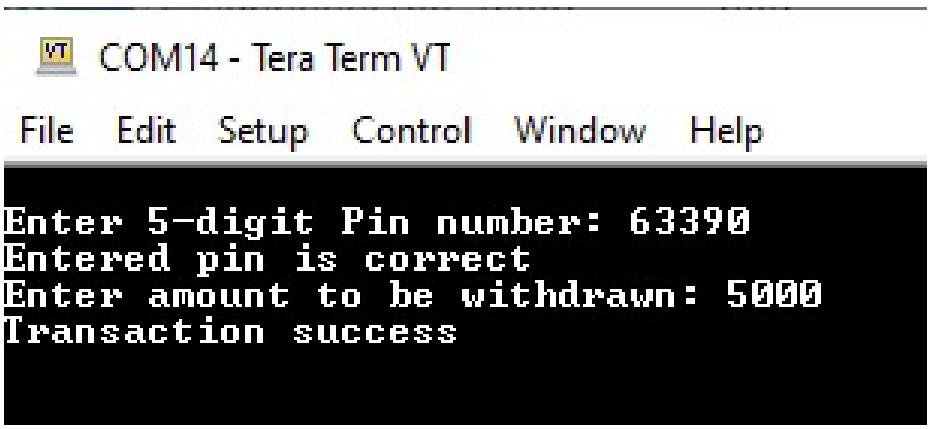


Figure 8: Correct PIN entered, valid withdrawal amount – "Transaction success"

Test Scenario	Expected Result
No withdrawal input within timeout	"Timeout!!"

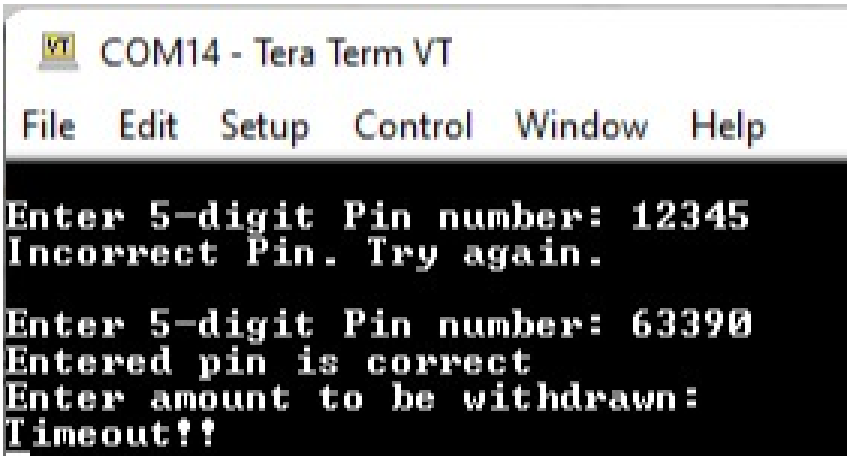


Figure 9: No input during withdrawal entry – "Timeout!!"

Test Scenario	Expected Result
No PIN input within timeout	"Timeout!!"

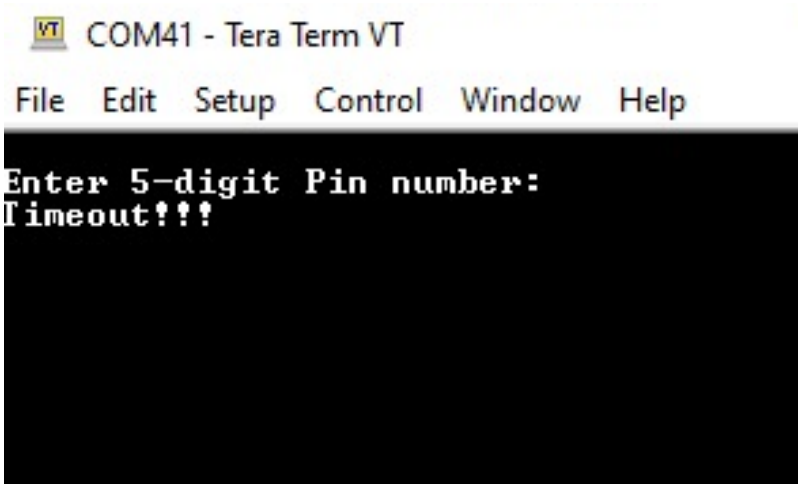


Figure 10: No input during PIN entry – "Timeout!!!"

Discussion and Conclusion

The Microprocessor-Based ATM Simulation project was a comprehensive exercise in embedded system development using the FRDM KL25Z platform. Through this project, we successfully implemented essential microcontroller skills such as GPIO configuration, UART communication, keypad scanning, software-based debouncing, and user input validation. The project also integrated timeout mechanisms to handle inactivity scenarios, which are critical for creating robust and user-friendly embedded applications.

Throughout the testing phase, the ATM system performed reliably across all intended use cases. Correct PIN entries were accurately authenticated, withdrawal amounts were correctly processed with appropriate validations, and timeout conditions were effectively handled, displaying corresponding feedback messages to the user. Special attention was given to the handling of mechanical keypad inputs, where debouncing proved essential in ensuring stable and accurate keypress detection. The UART interface with TeraTerm provided a clear and responsive communication channel between the system and the user, demonstrating effective serial data transmission and reception.

This project reinforced several important concepts in microcontroller programming, including real-time event handling, timeout implementations, and user-driven system interactions. Moreover, it emphasized the significance of error handling and system feedback in creating intuitive embedded interfaces. The design challenges encountered—such as synchronizing keypad inputs with UART outputs and implementing timeouts—offered valuable learning experiences that mirror practical issues found in real-world embedded system designs.

In conclusion, the successful completion of this project not only demonstrated technical proficiency in C programming and KL25Z hardware interfacing but also strengthened our ability to design, implement, and troubleshoot microprocessor-based systems. It has provided a solid foundation for more complex embedded system projects in the future, where real-time responsiveness, user input reliability, and system robustness are essential. Throughout the project, effective teamwork played a crucial role in completing the hardware setup, coding, and testing phases. Team members collaborated by distributing tasks based on strengths, such as debugging code and assembling the hardware connections. Challenges encountered during development included handling mechanical key bouncing, implementing accurate timeout mechanisms, and ensuring correct keypad scanning. These obstacles were overcome through testing and iterative improvements, enhancing our practical understanding of embedded system reliability and user interface responsiveness.